Querious AI

Implementation Plan

CSCI 5520 - AI-Driven Software Development

Edel Kiprop · Version 1.0 · October 2026

Derived from the Querious AI Software Design Document v1.0 (Final)

1 Implementation Overview

1.1 Implementation Objective

Querious AI will be implemented as a single-user, local web application: a React + TypeScript (Vite) front end, a Python FastAPI backend, an embedded read-only DuckDB dataset, a YAML semantic layer, a SQLite audit store, and a third-party LLM (Anthropic Claude, leading option) reached only through a provider interface. The implementation delivers the SDD v1.0 MVP: a business user loads a CSV or the sample dataset, asks a question in plain English, and receives an answer whose interpretation, plan, SQL, checks, verification support, provenance and corrections are all visible.

The plan preserves the SDD's central design decision, "the AI suggests, the system checks" (DD-01): the LLM only checks ambiguity and drafts a structured query plan; validation, compilation, read-only execution, result checks, trust scoring, provenance and explanation are deterministic, testable code. The core MVP is complete when US-01 to US-13 pass their SDD pass/fail checks. US-14 (direct LLM-to-SQL baseline comparison) is an optional research enhancement scheduled only after the core evaluation passes. No capability outside SDD Section 2.2 is added.

1.2 Source Design Artifacts

- **Software Design Document:** Querious AI SDD v1.0 (Final), Querious_AI_SDD_Final.pdf (21 pages, October 2026). Sections 3 (US-01 to US-14), 4 (RC-01 to RC-23), 5 (architecture) and 7 (DD-01 to DD-11, OQ/RISK items) are the primary inputs.
- **Interactive UI/UX prototype:** Querious prototype — Navy and Teal Workspace, built in Claude Design in place of Figma Make (SDD Section 6): https://claude.ai/artifact/Afcj1f7hvk7mhgHfFhNtQJ (page "6.2 Prototype (refined)"). Screen numbers 1–10 below refer to this prototype and Figure 2.
- **Architecture / experience-flow figures:** SDD Figure 1 (Section 5.2, architecture and bounded correction loop); SDD Figure 2 (Section 6.1, screens 1–10 and user roles); Table 6.2 (prototype flows A–F) and Table 6.3a (requirements alignment).
- **Other approved references:** GitHub repository https://github.com/chepkemboiedel-sudo/Querious-AI and GitHub Project "Querious AI Development" https://github.com/users/chepkemboiedel-sudo/projects/4 (Lab 2), the dev → staging → main branch rulesets, and the repository-root CLAUDE.md engineering policy (hard rules H1–H14 derived from the SDD).

1.3 Planning Assumptions & Scope Guardrails

- The final SDD v1.0 and the refined prototype are authoritative. Where the prototype and the SDD differ, the SDD wins (SDD 6.2); the prototype's in-browser rule-based planner is a stand-in for the LLM planner and is not reused as backend logic.
- MVP target: single user, local machine, current desktop Chrome, Firefox and Edge (RC-15, DD-07). No accounts, authentication, hosting, mobile layouts, cloud warehouses or production databases (SDD 2.3).
- Data is clean tabular CSV with a header row, English column names and up to 50 MB (ASSUMPTION-01). One fictional retail sample dataset (customers and orders, 2025–2026, as in the prototype) ships with the app and is reused for the gold question set.
- Read-only and no-raw-SQL rules (RC-01, RC-02) apply from the first executable code. The query plan (IR) supports only filters, aggregations, grouping, sorting, top-N, time bucketing and declared joins (OQ-02); anything else must fail honestly, not be added silently.
- Every pipeline step writes append-only audit events from Phase 2 onward so that replay, history and provenance (RC-09, RC-19, RC-22) are built from recorded evidence instead of being retrofitted later.

- Automated tests use a mocked LLM provider; only explicit evaluation runs (US-13, US-14) call the real API, to control cost and keep CI deterministic (RISK-03, RC-06).
- Unresolved before or during implementation: final LLM model (OQ-01), final gold-set size and labels (OQ-03) and initial sanity-check thresholds (RISK-02). These are recorded in Section 6 and must be decided by the human owner, not guessed by an agent.

Planning principle: Implementation agents may refine technical details inside the approved design, but they must not add product scope, weaken the read-only, data-minimization or no-fabrication rules, or change the IR, semantic-layer or audit formats without a documented human decision.

2 Phase Summary

Querious is divided into six phases. The sequence builds the trusted data and safety foundation first, then the end-to-end question pipeline, then clarification and correction, then the trust layer, then audit-driven history and follow-ups, and finally evaluation and hardening. Each phase ends in something that can be demonstrated in the browser or measured by a command.

Phase	Goal / Increment	Primary SDD Sources	Key Deliverables	Depends On	Candidate Issues
1	Foundation: Data Loading, Semantic Layer & Safety Base	US-01, US-02, US-12; RC-01, RC-05, RC-07, RC-08, RC-15, RC-16; DD-03, DD-07	Repo scaffold and CI; LLM provider interface; read-only CSV/sample loading; schema profile and fingerprint; validated semantic layer; append-only audit store; Start and Data overview screens	None	TMP-01 to TMP-07
2	Core Question Pipeline (happy path)	US-03; RC-02, RC-03, RC-06, RC-10 to RC-14; DD-01, DD-02, DD-04	IR schema; LLM planner; validator with capability gate; deterministic compiler; safe executor; templated summary; question API with stages; Ask, Working and Answer screens	Phase 1	TMP-08 to TMP-15
3	Clarification, Self-Correction & Honest Failure	US-04, US-06, US-07; RC-03, RC-04, RC-12, RC-20; DD-05, DD-06	Ambiguity check and alternatives; Clarify screen; base result checks; bounded correction loop; early rejection and service errors; Honest failure screen	Phase 2	TMP-16 to TMP-21

Phase	Goal / Increment	Primary SDD Sources	Key Deliverables	Depends On	Candidate Issues
4	Trust & Verification Layer	US-05, US-11; RC-17, RC-18, RC-19, RC-20; DD-08, DD-09	Risk classifier and check registry; Medium/High checks; deterministic trust policy; provenance builder; capability and risk preview; Verification support and Data used cards	Phases 2–3	TMP-22 to TMP-27
5	Audit, History, Replay & Follow-Ups	US-06, US-08, US-09, US-10; RC-09, RC-21, RC-22; DD-10	Complete RC-09 audit record; correction replay; history with reopen; answer feedback; schema-validated follow-ups	Phases 2–4	TMP-28 to TMP-32
6	Evaluation & MVP Hardening (US-14 optional)	US-13 (core), US-14 (optional); RC-14, RC-17 to RC-23; DD-11; OQ-03, OQ-04	Gold question set; one-command evaluation report; accessibility and acceptance pass; optional isolated baseline runner	Phases 1–5 (gold set can start in Phase 2)	TMP-33 to TMP-36

3 Detailed Implementation Phases

3.1 Phase 1 - Foundation: Data Loading, Semantic Layer & Safety Base

Goal

A user can start the app, load a valid CSV (or the sample dataset) into a read-only embedded database, and see its tables, column types, example values and steward-defined business terms. The shared foundations that every later phase depends on exist: the provider interface, the semantic-layer loader and the append-only audit store.

Source Requirements / Design References

US-01, US-02, US-12; RC-01, RC-05, RC-07, RC-08, RC-09, RC-15, RC-16; DD-03, DD-07; ASSUMPTION-01, OQ-05; SDD Figure 1 (data layer, semantic layer, audit store, provider boundary); prototype screens 1 (Start) and 2 (Data overview).

Deliverables

- Repository scaffold: backend/ (FastAPI, Python 3.12, pytest) and frontend/ (React + TypeScript via Vite), with a GitHub Actions workflow that runs backend and frontend tests on every pull request into dev.
- LLM provider interface module (the only place a model SDK is imported) with the API key read from a server-side environment variable and a mock provider for tests (RC-08, RC-16).
- CSV upload and sample-dataset loading into DuckDB opened read-only, with validation of file type, header row and the 50 MB limit (US-01, RC-01).
- Schema profile (tables, columns, types, optional example values off by default) and a dataset fingerprint used later by audit and provenance (US-02, RC-07, RC-09).

- semantic_layer.yaml format and loader that validates terms, metrics and declared joins against the schema at load time (US-12, RC-05, DD-03).
- SQLite audit store with an append-only event model that later phases extend (RC-09, DD-10).
- Start and Data overview screens with the navy sidebar navigation (screens 1–2).

Phase Acceptance Criteria

- A valid CSV up to 50 MB loads and its table and columns are displayed within 10 seconds; a non-CSV or header-less file is rejected with a clear message (US-01).
- The data overview lists every table and column with its type and every semantic-layer term with its plain-English description (US-02).
- A semantic-layer definition that refers to a missing column is rejected at load time with an error naming the problem (US-12).
- Any attempt to write through the dataset connection fails; the API key never appears in front-end code, logs or the repository (RC-01, RC-08).

Dependencies & Sequence Notes

No application prerequisite; this phase is first. The audit event model, semantic-layer format and dataset fingerprint must be settled here because Phases 2–5 build on them. The Start/Data overview screens can be built in parallel with the backend once the dataset and schema API shapes are agreed. Lab 2 Issues #3 (scaffold) and #4 (load a CSV) are reused for TMP-01 and TMP-03 rather than duplicated.

Relevant UI/UX Artifacts

Screen 1 Start (upload card with type/size/header checks, sample-dataset button, file-error state) and screen 2 Data overview (tables, column types, example values, read-only terms); prototype flow A in SDD Table 6.2.

Candidate GitHub Issues

- Scaffold FastAPI backend, React frontend, pytest and CI (TMP-01, existing #3)
- Implement LLM provider interface with server-side key handling (TMP-02)
- Load a CSV or the sample dataset into read-only DuckDB (TMP-03, existing #4)
- Profile the schema and compute a dataset fingerprint (TMP-04)
- Load and validate semantic_layer.yaml (TMP-05)
- Create the SQLite audit store with append-only events (TMP-06)
- Build the Start and Data overview screens (TMP-07)

3.2 Phase 2 - Core Question Pipeline (happy path)

Goal

A user can type a clear, supported question and receive an answer as a results table, a plain-English summary and a simple chart, produced by the full plan → validate → compile → execute pipeline, with the current stage shown while it works.

Source Requirements / Design References

US-03; RC-02, RC-03, RC-05, RC-06, RC-07, RC-10, RC-11, RC-12, RC-13, RC-14; DD-01, DD-02, DD-04; OQ-02; SDD Figure 1 (planner, validator, compiler, safe executor, explanation builder); prototype screens 3 (Ask), 5 (Working) and 6 (Answer).

Deliverables

- Pydantic query-plan (IR) schema covering exactly the OQ-02 operations (DD-02).
- LLM query planner that sends only schema metadata, terms and the question through the provider interface and parses the response into the IR (RC-02, RC-07).
- Plan validator: schema, terms, declared joins and the capability gate for unsupported operations (RC-05, RC-20).
- Deterministic SQLGlot compiler producing a single read-only SELECT; identical plans compile to identical SQL (RC-01, RC-06).
- Safe executor with a read-only connection, 15-second timeout and 5,000-row limit (RC-10).
- Templated summary built from the validated plan and actual results, never from free LLM text (RC-03, DD-04).
- Question API that reports the understanding, planning, checking and running stages and writes audit events (RC-09, RC-11).
- Ask, Working and Answer screens; every chart shown with its underlying table (RC-13, RC-14).

Phase Acceptance Criteria

- A clear gold-set question returns a table and summary backed by an executed query; no number appears without an executed result (US-03, RC-03).
- LLM output that is not a valid IR is never executed; any non-SELECT or multi-statement SQL is rejected before execution (RC-01, RC-02).
- The same validated plan compiled twice produces byte-identical SQL (RC-06).
- The interface shows the current stage while working; a 16-second query is stopped by the timeout and more than 5,000 rows are never sent to the interface (RC-10, RC-11).

Dependencies & Sequence Notes

Requires Phase 1 (provider interface, read-only dataset, schema profile, semantic layer, audit store). The IR schema (TMP-08) is the shared contract for the planner, validator, compiler, risk classifier and trust policy, so it is built and human-reviewed before them. Planner prompt work (TMP-09) and the compiler (TMP-11) can proceed in parallel once the IR is fixed. Front-end screens can use a mocked question API in parallel with backend work. The Lab 2 sub-issues for the IR model and SQLGlot compilation are reused as TMP-08 and TMP-11.

Relevant UI/UX Artifacts

Screen 3 Ask (“Ask your data” card, dataset-based suggestions, can/can’t scope hint), screen 5 Working (four named stages, “Nothing has been run on your data yet”) and screen 6 Answer (headline, “How I read your question”, chart and “View the underlying table”).

Candidate GitHub Issues

- Define the query-plan IR schema in Pydantic (TMP-08, existing sub-issue)
- Implement the LLM query planner (TMP-09)
- Implement the plan validator and capability gate (TMP-10)
- Compile validated plans to one read-only SELECT with SQLGlot (TMP-11, existing sub-issue)
- Implement the safe executor with timeout and row limit (TMP-12)
- Build the templated plain-English summary (TMP-13)
- Expose the question API with stage progress and audit events (TMP-14)
- Build the Ask, Working and Answer screens (TMP-15)

3.3 Phase 3 - Clarification, Self-Correction & Honest Failure

Goal

Ambiguous questions are clarified before anything runs, failed plans or failed checks are corrected automatically within three attempts, and questions that cannot be answered safely end in an honest failure with no number shown.

Source Requirements / Design References

US-04, US-06, US-07; RC-03, RC-04, RC-12, RC-20; DD-05, DD-06; RISK-01, RISK-02; SDD Figure 1 (dashed correction loop); prototype screens 4 (Clarify), 5 (Working with “Attempt n of 3”) and 8 (Can’t answer); Table 6.2 flows B, C and E.

Deliverables

- Ambiguity check that returns two or three schema-valid alternatives with their measure, grouping or time-period definitions (US-04, RC-20).
- Clarify screen: no option preselected, Continue disabled until a choice, “None of these, rephrase” runs nothing; the choice is recorded and revalidated.
- Base result sanity checks: empty result, impossible percentages, unexpected row multiplication (RC-03, RISK-02).
- Bounded correction loop that sends the exact validation error, database error or failed check back to the planner, at most three planning attempts (RC-04, DD-05).
- Early rejection of known unsafe, forecasting or missing-data requests before compilation, and a clear service error when the LLM is unavailable or rate-limited (US-07, RC-12, RC-20).
- Honest failure screen and AI-service-unavailable state with reason, what was tried and schema-valid rephrasing suggestions.

Phase Acceptance Criteria

- Each ambiguous gold-set case shows two or three alternatives before any query runs; Continue stays disabled until one is chosen; “None of these, rephrase” executes nothing (US-04).
- An injected first-attempt schema failure is recovered within three total planning attempts; failed attempts never display a result (US-06).
- Unsafe, forecasting and missing-data requests stop before compilation, and an execution spy records zero executions for them; after three failures no answer or score is shown (US-07).
- With the LLM unavailable, the user sees a service error with Try again rather than a hang or partial answer (RC-12).

Dependencies & Sequence Notes

Requires the Phase 2 pipeline, validator and audit events. The correction loop depends on the base checks (TMP-18) and validator errors being precise. Clarification (TMP-16, TMP-17) and the failure path (TMP-20, TMP-21) can be built in parallel. The Lab 2 sub-issue for retrying failed plans is reused as TMP-19.

Relevant UI/UX Artifacts

Screen 4 Clarify (radio options stating each definition, “No query has been run yet”), screen 5 Working (“Attempt n of 3”), screen 8 Can’t answer (“What I tried”, “No numbers are shown”, attempt list, suggestions) and the AI-service-unavailable state.

Candidate GitHub Issues

- Implement the ambiguity check with schema-valid alternatives (TMP-16)

- Build the Clarify screen (TMP-17)
- Implement base result sanity checks (TMP-18)
- Implement bounded self-correction with exact error feedback (TMP-19, existing sub-issue)
- Implement early rejection, honest failure and LLM service errors (TMP-20)
- Build the Honest failure and service-unavailable screens (TMP-21)

3.4 Phase 4 - Trust & Verification Layer

Goal

Every released answer carries deterministic verification support (High, Medium or Low with reasons), a risk class with its mandatory checks, and complete provenance; the user sees operations and risk before the answer query runs.

Source Requirements / Design References

US-05, US-07, US-11; RC-17, RC-18, RC-19, RC-20, RC-23 (helper SELECTs); DD-08, DD-09; RISK-04; prototype screens 5 (preview), 6 and 7 (Verification support and Data used cards, “How I got this”); Table 6.2 flows B and D.

Deliverables

- Deterministic risk classifier over the validated IR and a small versioned registry of checks per class (RC-18, DD-09).
- Medium and High checks: aggregate type, null handling, grouping grain, date boundaries, declared-key/cardinality, row multiplication and independent reconciliation, using safety-checked helper SELECTs (RC-18, RC-23).
- Trust policy computing High, Medium, Low or Not available from recorded signals, with fixed signal fixtures; no percentages (RC-17, DD-08).
- Provenance builder: sources, fingerprint, columns, predicates, date boundaries, rows considered, per-source counts for joins, returned rows and truncation (RC-19).
- Capability and risk preview shown after validation and before execution (RC-20).
- Verification support and Data used cards directly below the answer, with “How I got this” for risk-grouped checks, plan and SQL (US-05, RC-13).

Phase Acceptance Criteria

- Fixed low, medium and high plan fixtures receive the RC-18 class and all mandatory checks for that class run before release; a failed join-cardinality check blocks the answer and triggers correction (US-11).
- Fixed signal fixtures produce exactly the RC-17 level; failed or unavailable mandatory checks show “Not available” instead of a score; no confidence percentage appears anywhere (US-05).
- Rows considered is computed from execution metadata, never from result length, and matches the audit record (RC-19).
- Every answer shows trust level and reasons, interpretation, terms, plan, risk class, executed SQL, checks, sources, columns, rows considered and filters (US-05).

Dependencies & Sequence Notes

Requires the validated IR and validator (Phase 2), base checks and the correction loop (Phase 3), because trust levels depend on correction count and check outcomes. The risk classifier (TMP-22) must exist before Medium/High checks (TMP-23) and the trust policy (TMP-24). The cards (TMP-27) follow the trust policy and provenance builder. Lab 2 Issue #6 is reused as TMP-27.

Relevant UI/UX Artifacts

Screen 5 preview line (operations and risk class), screen 6 Answer, and screen 7 evidence cards: “Verification support” (level, reasons, “not a probability” note) and “Data used” (sources, columns, rows considered, filters) directly below the answer, as in the final prototype layout (SDD Table 6.3b, last row).

Candidate GitHub Issues

- Implement the deterministic risk classifier and check registry (TMP-22)
- Implement Medium and High risk checks with helper SELECTs (TMP-23)
- Implement the deterministic trust policy with fixtures (TMP-24)
- Implement the provenance builder (TMP-25)
- Show the capability and risk preview before execution (TMP-26)
- Build the Verification support and Data used cards (TMP-27, existing #6)

3.5 Phase 5 - Audit, History, Replay & Follow-Ups

Goal

Every question leaves a complete audit record that can be replayed and reopened after a restart, users can rate answers, and up to three schema-validated follow-up questions keep exploration inside supported analysis.

Source Requirements / Design References

US-06, US-08, US-09, US-10; RC-09, RC-21, RC-22; DD-10; prototype screens 6 (follow-ups), 7 (Replay correction), 9 (History) and 10 (Feedback); Table 6.2 flows C and F.

Deliverables

- Audit record with every RC-09 field, including policy versions, signals, provenance and replay events.
- Correction replay derived only from immutable audit events, distinguishing “not executed”, “executed but failed checks” and “answer released” (RC-22).
- History list that survives restart; reopening shows the original answer, evidence and replay without rerunning queries (US-08).
- Correct / Flag an issue feedback stored with the matching audit record and visible on reopen (US-09).
- Up to three follow-up suggestions from schema/semantic-layer templates and the final plan, each validated and run as a new question (US-10, RC-21).

Phase Acceptance Criteria

- After restarting the app, previous questions appear in History and reopening one shows the original answer and full audit trail with no new execution (US-08).
- Replay shows each attempt number, failed stage, error, plan change and final outcome, identical on reopen (US-06, RC-22).
- A rating is stored with its audit record and shown when the answer is reopened (US-09).
- Missing-column and forecasting fixtures produce no unsupported suggestions; selecting a suggestion runs the full pipeline (US-10).

Dependencies & Sequence Notes

Requires the audit event model (Phase 1), events written by the pipeline (Phases 2–3) and the trust and provenance data (Phase 4). Replay, history and feedback depend on the completed audit record

(TMP-28); follow-ups (TMP-32) depend only on the validator and Answer screen and can run in parallel with them.

Relevant UI/UX Artifacts

Screen 6 follow-up chips, screen 7 “Replay correction” step-through, screen 9 History (status, support level, clarification choice, rating) and screen 10 feedback buttons.

Candidate GitHub Issues

- Complete the RC-09 audit record (TMP-28)
- Build the correction replay from audit events (TMP-29)
- Implement question history with reopen (TMP-30)
- Implement answer feedback (TMP-31)
- Generate up to three schema-validated follow-up questions (TMP-32)

3.6 Phase 6 - Evaluation & MVP Hardening

Goal

The core MVP is measured against the SDD with one command and its user stories are demonstrated as passing; known limitations are documented. The optional US-14 baseline comparison is built only after the core evaluation passes.

Source Requirements / Design References

US-13 (core), US-14 (optional); RC-14, RC-15, RC-17 to RC-23; DD-11; OQ-03, OQ-04; SDD Section 7.2 evaluation plan; all prototype flows A–F.

Deliverables

- Frozen sample dataset and gold question set of 40–50 questions labeled answerable, ambiguous or unsupported, with gold plans, results and risk classes (OQ-03).
- One-command evaluation harness reporting execution accuracy, clarification precision/recall, correction success, repeated-run consistency, median/p90 latency, calls per question and RC-17 to RC-22 fixture results by risk class (US-13).
- Accessibility and acceptance pass: keyboard navigation, contrast, screen-reader labels, status not by color alone, every chart with its table, supported browsers (RC-14, RC-15).
- Optional: isolated developer-only baseline runner comparing direct LLM-to-SQL with Querious on identical frozen inputs (US-14, RC-23, OQ-04).

Phase Acceptance Criteria

- The evaluation command runs end to end and produces the US-13 report; the 80% answerable-accuracy target is reported as measured, not assumed.
- Every US-01 to US-13 pass/fail check is demonstrated or documented as passing, with known non-blocking limitations listed.
- No exposed secret, write path or raw-SQL execution path exists in the user-facing app.
- If built, the baseline runner cannot be reached from the business-user API and never shows baseline output as a trusted answer (US-14).

Dependencies & Sequence Notes

Depends on Phases 1–5 for the harness and acceptance pass. Writing the gold set (TMP-33) needs the IR from Phase 2 and can run in parallel from then on; it requires human labeling. US-14 (TMP-36)

starts only after TMP-34 passes. Defect fixes may return work to earlier areas, but no new scope is introduced during hardening. Lab 2 Issue #7 is reused as TMP-34.

Relevant UI/UX Artifacts

All prototype flows A–F (SDD Table 6.2) as the acceptance reference; no new screens. US-13 and US-14 are developer commands with no business-user UI.

Candidate GitHub Issues

- Build the frozen sample dataset and gold question set (TMP-33)
- Implement the one-command evaluation harness and report (TMP-34, existing #7)
- Run the accessibility and SDD acceptance pass (TMP-35)
- Optional: build the isolated baseline comparison runner (TMP-36)

4 Cross-Phase Dependencies & Implementation Sequence

The critical path runs through the shared contracts: scaffold (TMP-01) → read-only dataset and schema profile (TMP-03, TMP-04) → semantic layer (TMP-05) → IR schema (TMP-08) → validator (TMP-10) → compiler (TMP-11) → safe executor (TMP-12) → question API (TMP-14) → correction loop (TMP-19) → risk classifier and trust policy (TMP-22, TMP-24) → complete audit record (TMP-28) → evaluation harness (TMP-34). Delays on these items delay everything after them.

Four shared prerequisites are used by many later Issues and must be stable, tested and human-reviewed before dependents start: the provider interface (TMP-02), the semantic-layer format (TMP-05), the append-only audit event model (TMP-06) and the IR schema (TMP-08). Changing any of them later requires a human decision because it ripples through validation, trust, provenance and replay.

Parallel work: front-end screens can be built against a mocked API once each API shape is agreed; planner prompting (TMP-09) can proceed alongside the compiler (TMP-11); clarification and failure paths in Phase 3 are independent; follow-ups (TMP-32) can run alongside history and replay; and the gold set (TMP-33) can be written from Phase 2 onward.

Sequencing rules agents must not violate: no executable query path exists before the read-only executor and validator are in place; no UI shows a trust level before the RC-17 policy and its fixtures exist; replay is never generated by a model call; and US-14 work does not start until the core US-13 evaluation passes.

4.1 Dependency / Sequence Table

Item / Capability	Must Follow	Reason	Can Run in Parallel With
Question pipeline (planner, compiler, executor)	IR schema (TMP-08), semantic layer (TMP-05), read-only dataset (TMP-03)	Every pipeline step reads or produces the IR and must respect declared joins and read-only access	Start and Data overview screens (TMP-07)
Executable SQL path	Validator (TMP-10) and compiler (TMP-11)	RC-01/RC-02: nothing runs unless it is a validated, compiled single SELECT	Templated summary design (TMP-13)
Bounded correction loop (TMP-19)	Validator errors (TMP-10) and base checks (TMP-18)	Retries need precise error signals to send back (DD-05)	Clarify screen (TMP-17)

Item / Capability	Must Follow	Reason	Can Run in Parallel With
Trust policy (TMP-24)	Risk checks (TMP-23) and correction loop (TMP-19)	RC-17 levels depend on check outcomes and correction count	Provenance builder (TMP-25)
Verification and Data used cards (TMP-27)	Trust policy (TMP-24) and provenance (TMP-25)	Cards must show recorded evidence, not placeholders	Follow-up suggestions (TMP-32)
Replay and history (TMP-29, TMP-30)	Complete audit record (TMP-28)	Both are views of immutable audit events (RC-22, US-08)	Answer feedback (TMP-31)
Evaluation harness (TMP-34)	Gold set (TMP-33) and Phases 2–5	Metrics and RC-17 to RC-22 fixtures need the full pipeline	Accessibility pass (TMP-35)
Baseline runner (TMP-36, optional)	Passing core evaluation (TMP-34)	US-14 is a research enhancement after the core MVP (DD-11)	Final defect fixes

5 GitHub Issue Decomposition Guidance

Each candidate item becomes one focused GitHub Issue that an implementation agent can complete in a single branch and pull request into dev. Issues reference the SDD and this plan by ID instead of copying them, and carry enough context that the agent does not need to redesign anything.

- **Outcome-oriented title:** states the result, for example “Compile validated plans to one read-only SELECT with SQLGlot”, not “Work on SQL”.
- **Goal and context:** two or three sentences on what the user or system gains and where it sits in the pipeline (SDD Figure 1).
- **Traceability:** implementation phase, TMP ID, and the US-##, RC-## and DD-## items it satisfies.
- **Acceptance criteria / definition of done:** objective checkboxes reusing SDD pass/fail checks where they apply; “done” means merged into dev with tests passing.
- **Design references:** prototype screen numbers and link, SDD figure or table, and images from docs/issue-assets/ when the Issue has UI.
- **Dependencies and constraints:** prerequisite Issues (also set as GitHub “blocked by” where supported), relevant CLAUDE.md hard rules, likely files or modules, and explicit non-goals.
- **Verification:** the tests or fixtures required (unit, API, UI, execution spy, signal fixtures), using the mocked LLM provider unless the Issue is an evaluation run.
- **Size:** large enough to produce a demonstrable or testable result, small enough for one focused implementation and review cycle (about one to three days of work); larger work is split into sub-issues.

5.1 Candidate Issue Inventory

ID*	Issue Title	Phase	SDD Sources	Depends On	Notes / Visual Context
TMP-01	Scaffold FastAPI backend, React frontend, pytest and CI	1	RC-08, RC-15; DD-07	None	Reuses Lab 2 Issue #3; SDD 5.1 technology table.
TMP-02	Implement LLM provider interface with server-side key handling	1	RC-08, RC-12, RC-16; DD-01	TMP-01	Mock provider for tests; Figure 1 provider boundary.

ID*	Issue Title	Phase	SDD Sources	Depends On	Notes / Visual Context
TMP-03	Load a CSV or the sample dataset into read-only DuckDB	1	US-01; RC-01; ASSUMPTION-01	TMP-01	Reuses Lab 2 Issue #4; screen 1 Start, file-error state.
TMP-04	Profile the schema and compute a dataset fingerprint	1	US-02; RC-07, RC-09; OQ-05	TMP-03	Example values off by default; screen 2.
TMP-05	Load and validate semantic_layer.yaml	1	US-12; RC-05; DD-03	TMP-04	Terms, metrics, declared joins; screen 2 terms list.
TMP-06	Create the SQLite audit store with append-only events	1	RC-09, RC-22; DD-10	TMP-01	Event model reused by Phases 2–5.
TMP-07	Build the Start and Data overview screens	1	US-01, US-02, US-12; RC-13, RC-14	TMP-03, TMP-05	Screens 1–2, navy sidebar; prototype flow A.
TMP-08	Define the query-plan IR schema in Pydantic	2	RC-02; DD-02; OQ-02	TMP-01	Reuses Lab 2 sub-issue; shared contract.
TMP-09	Implement the LLM query planner	2	US-03; RC-02, RC-06, RC-07	TMP-02, TMP-05, TMP-08	Schema-only prompts; Figure 1 planner.
TMP-10	Implement the plan validator and capability gate	2	RC-02, RC-05, RC-20; US-07	TMP-05, TMP-08	Precise errors feed TMP-19.
TMP-11	Compile validated plans to one read-only SELECT with SQLGlot	2	RC-01, RC-02, RC-06	TMP-08, TMP-10	Reuses Lab 2 sub-issue; determinism test.
TMP-12	Implement the safe executor with timeout and row limit	2	RC-01, RC-10	TMP-03, TMP-11	15 s timeout, 5,000 rows.
TMP-13	Build the templated plain-English summary	2	RC-03, RC-13; DD-04	TMP-12	Screen 6 headline and summary.
TMP-14	Expose the question API with stage progress and audit events	2	RC-09, RC-11	TMP-06, TMP-09 to TMP-13	Four stages; screen 5.
TMP-15	Build the Ask, Working and Answer screens	2	US-03; RC-11, RC-13, RC-14	TMP-07, TMP-14	Screens 3, 5, 6; chart plus table.
TMP-16	Implement the ambiguity check with schema-valid alternatives	3	US-04; RC-20; DD-06; RISK-01	TMP-05, TMP-09	Two or three described meanings.
TMP-17	Build the Clarify screen	3	US-04; RC-14	TMP-15, TMP-16	Screen 4; prototype flow B.
TMP-18	Implement base result sanity checks	3	RC-03; RISK-02	TMP-12	Empty, impossible %, row multiplication.
TMP-19	Implement bounded self-correction with exact error feedback	3	US-06; RC-04; DD-05	TMP-10, TMP-14, TMP-18	Reuses Lab 2 sub-issue; flow C.

ID*	Issue Title	Phase	SDD Sources	Depends On	Notes / Visual Context
TMP-20	Implement early rejection, honest failure and LLM service errors	3	US-07; RC-03, RC-12, RC-20	TMP-10, TMP-19	Execution spy; flow E.
TMP-21	Build the Honest failure and service-unavailable screens	3	US-07; RC-12, RC-13	TMP-15, TMP-20	Screen 8 and Try again state.
TMP-22	Implement the deterministic risk classifier and check registry	4	US-11; RC-18; DD-09	TMP-10, TMP-18	Versioned registry.
TMP-23	Implement Medium and High risk checks with helper SELECTs	4	US-11; RC-18, RC-23	TMP-22	Join cardinality, reconciliation.
TMP-24	Implement the deterministic trust policy with fixtures	4	US-05; RC-17; DD-08	TMP-19, TMP-23	No percentages; Not available state.
TMP-25	Implement the provenance builder	4	US-05; RC-19	TMP-12, TMP-23	Rows considered from metadata.
TMP-26	Show the capability and risk preview before execution	4	US-07, US-11; RC-20	TMP-15, TMP-22	Screen 5 preview line.
TMP-27	Build the Verification support and Data used cards	4	US-05; RC-13, RC-17, RC-19	TMP-24, TMP-25	Reuses Lab 2 Issue #6; screen 7 below the answer.
TMP-28	Complete the RC-09 audit record	5	RC-09	TMP-06, TMP-24, TMP-25	All RC-09 fields and versions.
TMP-29	Build the correction replay from audit events	5	US-06; RC-22	TMP-28	Screen 7 Replay correction.
TMP-30	Implement question history with reopen	5	US-08	TMP-28	Screen 9; no rerun on reopen.
TMP-31	Implement answer feedback	5	US-09	TMP-28	Screen 10 Correct / Flag an issue.
TMP-32	Generate up to three schema-validated follow-up questions	5	US-10; RC-21	TMP-10, TMP-15	Screen 6 follow-up chips.
TMP-33	Build the frozen sample dataset and gold question set	6	US-13; OQ-03	TMP-08	40–50 labeled questions; human labeling.
TMP-34	Implement the one-command evaluation harness and report	6	US-13; RC-17 to RC-22	TMP-33, Phases 2–5	Reuses Lab 2 Issue #7.
TMP-35	Run the accessibility and SDD acceptance pass	6	RC-14, RC-15; US-01 to US-13	Phases 1–5	All prototype flows A–F.

ID*	Issue Title	Phase	SDD Sources	Depends On	Notes / Visual Context
TMP-36	Optional: build the isolated baseline comparison runner	6	US-14; RC-23; DD-11; OQ-04	TMP-34	Developer-only; not in user API.

**TMP IDs are temporary. Lab 2 Issues #3, #4, #6 and #7 and the Lab 2 sub-issues are reused for TMP-01, TMP-03, TMP-27, TMP-34, TMP-08, TMP-11 and TMP-19 and updated rather than duplicated; GitHub numbers replace the TMP IDs once the Issues are created.*

6 Risks, Open Decisions & Agent Guardrails

These items complement the SDD (Section 7.2) and the CLAUDE.md hard rules. Open decisions belong to the human owner; an agent that reaches one must stop, comment on the Issue and move it back to Ready instead of choosing.

Open decisions (human owner)

- **OD-1 LLM model (OQ-01):** the model is not chosen. Agents use the provider interface with the model name in configuration and must not hard-code a model or call another provider directly. The human owner selects the model after comparing candidates on the gold set.
- **OD-2 IR coverage (OQ-02):** limited to filters, aggregations, grouping, sorting, top-N, time bucketing and declared joins. Agents must not add window functions, subqueries or free SQL to make a question pass; unsupported questions fail honestly.
- **OD-3 Gold set and dataset (OQ-03):** the human owner approves the frozen sample dataset and labels each question (answerable, ambiguous, unsupported) with gold results; agents may draft candidates but not finalize labels or expected answers.
- **OD-4 Example values to the LLM (OQ-05):** off by default; agents must not enable it or send row values for any reason.
- **OD-5 Initial check thresholds (RISK-02):** the starting set of sanity and risk checks and their thresholds (for example, what counts as unexpected row multiplication) are approved by the human owner in TMP-18 and TMP-23 before the trust policy depends on them.
- **OD-6 Stage progress transport (decided):** the owner chose polling (about every 500 ms) for RC-11 stages; it is recorded in docs/ARCHITECTURE.md and TMP-14 must follow it. The owner also decided that one dataset is active at a time and is kept locally between restarts.

Implementation risks

- **Audit retrofit risk:** if early phases skip audit events, replay and history later require rework. Mitigation: append-only events from TMP-06 and TMP-14 onward.
- **Contract churn:** changes to the IR, semantic layer or audit format ripple through many Issues. Mitigation: human review before dependents start; changes need an ADR.
- **LLM cost and latency during development (RISK-03):** mitigated by the mock provider in tests and by tracking calls per question in TMP-34.
- **Over-trust (RISK-04) and check errors (RISK-02):** trust levels must report evidence, not correctness; false positives are measured in the evaluation report.
- **Prototype divergence:** the prototype's browser planner is not backend logic; the SDD is authoritative where they differ.

Agent guardrails

- Never execute LLM output as SQL or code; only validated, compiled single SELECT statements run, on a read-only connection (RC-01, RC-02).
- Never send data rows to the LLM, never expose the API key, and never call the real LLM from unit tests (RC-07, RC-08).

- Never show a number, summary, trust level or percentage that is not derived from executed results and recorded signals (RC-03, RC-17).
- Never exceed three planning attempts or generate replay or provenance with a model call (RC-04, RC-19, RC-22).
- Do not add scope outside SDD 2.2: no writing data, multi-user accounts, dashboards (FUT-01), forecasting or cloud databases.
- Work only through feature branches and pull requests into dev; staging and main promotions are approved and merged by the human owner.

7 Handoff to GitHub Project

All candidate Issues are created by Claude Code from this plan in the AiDev environment, added to the GitHub Project "Querious AI Development" (https://github.com/users/chepkemboiedel-sudo/projects/4) and linked to the repository. Lab 2 Issues are reused and updated rather than duplicated. From this point the GitHub Project and Issues are the authoritative source for detailed tasks, acceptance criteria, sequencing, status and day-to-day tracking; this plan remains the explanation of phases, dependencies and intent.

Project fields carry the plan's metadata: **Phase** (P1 Foundation & data through P6 Evaluation & hardening), **Sequence** (the TMP number, giving build order), **Status** (Backlog, Ready, In Progress, Agent Review, Done in Dev, Staging Candidate, Released), **Priorities** (P0 core MVP, P1 polish and hardening, P2 optional US-14), **Area** (Frontend, Backend API, Pipeline, Data, Semantic layer, Evaluation, DevOps), **Executor** (AiDev, Human, Pair), **SDD refs** (US/RC/DD IDs) and **Human Review** (Pending, Reviewed). Dependencies are recorded as GitHub "blocked by" relationships where supported and in each Issue's Dependencies section.

Every AI-created Issue starts as Human Review = Pending and Status = Backlog. The human owner changes an Issue to Reviewed only after checking its scope, SDD traceability, acceptance criteria, design references, dependencies and verification expectations, and moves it to Ready only when it is reviewed and its prerequisites are done. Phase 1 Issues are the first to become Ready.

7.1 Final Readiness Checklist

- Every MVP user story (US-01 to US-13) is represented by at least one phase and candidate Issue; optional US-14 is TMP-36.
- Every phase has concrete deliverables and objective acceptance criteria tied to SDD pass/fail checks.
- Dependencies, the critical path and sequencing rules are explicit (Section 4 and 4.1), so an agent does not have to guess the order of high-risk work.
- Relevant prototype screens, SDD figures and tables are identified for every phase, and UI Issues include images from docs/issue-assets/.
- The candidate Issue inventory (Section 5.1) is detailed enough to create GitHub Issues without redesigning the project, and open decisions are recorded in Section 6.
- The GitHub Project has the Phase, Sequence and Human Review fields configured before Issue creation; every Issue starts as Pending.